

```

import os
import json
import asyncio
import logging
import time

from datetime import datetime, timedelta
from telegram import Update, InlineKeyboardButton, InlineKeyboardMarkup
from telegram.ext import Application, CommandHandler, ContextTypes, MessageHandle
import requests

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

TOKEN = os.environ.get("TELEGRAM_BOT_TOKEN", "")
SOLSCAN_API = os.environ.get("SOLSCAN_API_KEY", "")
ETHERSCAN_API = os.environ.get("ETHERSCAN_API_KEY", "")
ADMIN_ID = int(os.environ.get("ADMIN_ID", "0"))

# Load VIP wallets
def load_vip_wallets():
    try:
        with open("vip_wallets.json", "r") as f:
            return json.load(f)
    except:
        return []

# Load CEX wallets
def load_cex_wallets():
    try:
        with open("cex_wallets.json", "r") as f:
            return json.load(f).get("cex_wallets", [])
    except:
        return []

# Load users
def load_users():
    try:
        with open("users.json", "r") as f:
            return json.load(f)
    except:
        return {}

# Save users
def save_users(users):

```

```

        with open("users.json", "w") as f:
            json.dump(users, f, indent=2)

# Load seen transactions
def load_seen_txs():
    try:
        with open("seen_txs.json", "r") as f:
            return json.load(f)
    except:
        return []

# Save seen transactions
def save_seen_txs(txs):
    with open("seen_txs.json", "w") as f:
        json.dump(txs[-5000:], f)

# Recent buys tracker for signals
recent_buys = {}

def get_solana_transactions(address):
    try:
        headers = {"token": SOLSCAN_API} if SOLSCAN_API else {}
        url = f"https://public-api.solscan.io/account/transactions?account={address}"
        r = requests.get(url, headers=headers, timeout=10)
        if r.status_code == 200:
            return r.json()
    except:
        pass
    return []

def get_solana_tx_detail(signature):
    try:
        headers = {"token": SOLSCAN_API} if SOLSCAN_API else {}
        url = f"https://public-api.solscan.io/transaction/{signature}"
        r = requests.get(url, headers=headers, timeout=10)
        if r.status_code == 200:
            return r.json()
    except:
        pass
    return None

def get_token_info_gmgn(token_address):
    try:
        url = f"https://gmgn.ai/defi/quotation/v1/tokens/sol/{token_address}"
        r = requests.get(url, timeout=10)
        if r.status_code == 200:
            data = r.json()

```

```

        token = data.get("data", {}).get("token", {})
        return {
            "name": token.get("name", "Unknown"),
            "symbol": token.get("symbol", "???"),
            "market_cap": token.get("market_cap", 0),
            "holder_count": token.get("holder_count", 0),
            "fresh_wallet_ratio": token.get("smart_degen_count", 0),
        }
    except:
        pass
    return None

def get_dexscreener_info(token_address):
    try:
        url = f"https://api.dexscreener.com/latest/dex/tokens/{token_address}"
        r = requests.get(url, timeout=10)
        if r.status_code == 200:
            data = r.json()
            pairs = data.get("pairs", [])
            if pairs:
                p = pairs[0]
                return {
                    "name": p.get("baseToken", {}).get("name", "Unknown"),
                    "symbol": p.get("baseToken", {}).get("symbol", "???"),
                    "price_usd": p.get("priceUsd", "0"),
                    "market_cap": p.get("marketCap", 0),
                    "volume_24h": p.get("volume", {}).get("h24", 0),
                    "liquidity": p.get("liquidity", {}).get("usd", 0),
                    "price_change_5m": p.get("priceChange", {}).get("m5", 0),
                    "created_at": p.get("pairCreatedAt", 0),
                }
            else:
                return {}
        else:
            return None
    except:
        pass
    return None

def get_rugcheck_info(token_address):
    try:
        url = f"https://api.rugcheck.xyz/v1/tokens/{token_address}/report"
        r = requests.get(url, timeout=10)
        if r.status_code == 200:
            data = r.json()
            risks = data.get("risks", [])
            score = data.get("score", 0)
            mint_ok = not any("mint" in str(r).lower() for r in risks)
            freeze_ok = not any("freeze" in str(r).lower() for r in risks)
            return {
                "score": score,
            }
        else:
            return None
    except:
        pass
    return None

```

```

        "mint_disabled": mint_ok,
        "freeze_disabled": freeze_ok,
        "risks": risks[:3],
    }
except:
    pass
return None

def get_bubblemaps_info(token_address):
    try:
        url = f"https://api-legacy.bubblemaps.io/map-metadata?token={token_address}"
        r = requests.get(url, timeout=10)
        if r.status_code == 200:
            data = r.json()
            identified = data.get("identified_supply", {})
            bundle_pct = identified.get("percent_by_identified_entities", 0)
            return {"bundle_pct": round(bundle_pct, 1)}
    except:
        pass
    return None

def calculate_safety_score(rugcheck, dex, bubblemaps):
    score = 50
    if rugcheck:
        if rugcheck.get("mint_disabled"): score += 15
        if rugcheck.get("freeze_disabled"): score += 15
    if bubblemaps:
        bundle = bubblemaps.get("bundle_pct", 50)
        if bundle < 10: score += 15
        elif bundle < 20: score += 5
        else: score -= 10
    if dex:
        age_ms = dex.get("created_at", 0)
        if age_ms:
            age_hours = (time.time() * 1000 - age_ms) / 3600000
            if age_hours > 24: score += 5
    return min(100, max(0, score))

def format_number(n):
    try:
        n = float(n)
        if n >= 1_000_000: return f"${n/1_000_000:.1f}M"
        if n >= 1_000: return f"${n/1_000:.1f}K"
        return f"${n:.1f}"
    except:
        return "N/A"

```

```

def get_risk_emoji(score):
    if score >= 70: return "🟢"
    if score >= 40: return "🟡"
    return "🔴"

def get_bundle_emoji(pct):
    if pct < 10: return "🟢"
    if pct < 20: return "🟡"
    return "🔴"

async def send_signal_alert(app, user_id, token_address, buyers, wallet_names):
    dex = get_dexscreener_info(token_address)
    rugcheck = get_rugcheck_info(token_address)
    bubblemaps = get_bubblemaps_info(token_address)
    safety_score = calculate_safety_score(rugcheck, dex, bubblemaps)

    name = dex.get("name", "Unknown") if dex else "Unknown"
    symbol = dex.get("symbol", "???") if dex else "???"
    mcap = format_number(dex.get("market_cap", 0)) if dex else "N/A"
    volume = format_number(dex.get("volume_24h", 0)) if dex else "N/A"
    liquidity = format_number(dex.get("liquidity", 0)) if dex else "N/A"

    mint_str = "✅ Disabled" if (rugcheck and rugcheck.get("mint_disabled")) else
    freeze_str = "✅ Disabled" if (rugcheck and rugcheck.get("freeze_disabled"))

    bundle_pct = bubblemaps.get("bundle_pct", 0) if bubblemaps else 0
    bundle_str = f"{get_bundle_emoji(bundle_pct)} {bundle_pct}%"

    score_emoji = get_risk_emoji(safety_score)
    short_addr = f"{token_address[:6]}...{token_address[-4:]}"

    buyers_list = "\n".join([f"• {name}" for name in wallet_names])

    msg = f"""🔴🔴 *HIGH SIGNAL - {len(buyers)} WALLETS BOUGHT*

🌐 *{name}* (${symbol})
📍 CA: `{token_address}`

👥 *Tracked Buyers ({len(buyers)}) :*
{buyers_list}

📊 *Market Data:*
• MCap: {mcap}
• Volume 24h: {volume}
• Liquidity: {liquidity}

🔍 *Safety Analysis:*

```

- Mint Authority: {mint_str}
- Freeze Authority: {freeze_str}
- Bundle: {bundle_str}

 *SAFETY SCORE: {safety_score}/100* {score_emoji}

 *Links:*

- [Dexscreener] (https://dexscreener.com/solana/{token_address})
- [GMGN] (https://gmgn.ai/sol/token/{token_address})
- [Bubblemaps] (https://app.bubblemaps.io/sol/token/{token_address})
- [RugCheck] (https://rugcheck.xyz/tokens/{token_address}) ""

try:

```
await app.bot.send_message(user_id, msg, parse_mode="Markdown", disable_w
except Exception as e:
    logger.error(f"Error sending signal: {e}")
```

```
async def send_individual_alert(app, user_id, wallet_name, wallet_address, token_
    dex = get_dexscreener_info(token_address)
    name = dex.get("name", "Unknown") if dex else "Unknown"
    symbol = dex.get("symbol", "???" ) if dex else "???"
    short_wallet = f"{wallet_address[:6]}...{wallet_address[-4:]}"
```

msg = f""  *Wallet Activity*

 *{wallet_name or short_wallet}*

 Bought: *{name}* (\${symbol})

 Amount: *{amount_sol:.3f}* SOL*

 CA: `{token\_address}`

 [Dexscreener] (https://dexscreener.com/solana/{token_address}) | [GMGN] (https://gmgn.ai/sol/token/{token_address})

try:

```
await app.bot.send_message(user_id, msg, parse_mode="Markdown", disable_w
except Exception as e:
    logger.error(f"Error sending individual alert: {e}")
```

```
async def check_wallets(app):
```

```
    vip_wallets = load_vip_wallets()
```

```
    users = load_users()
```

```
    seen_txs = load_seen_txs()
```

```
    now = time.time()
```

```
    for wallet in vip_wallets:
```

```
        address = wallet.get("address", "")
```

```
        wallet_name = wallet.get("name", address[:8])
```

```
        if not address:
```

```

        continue

txs = get_solana_transactions(address)
if not txs:
    continue

for tx in txs[:5]:
    sig = tx.get("txHash", "")
    if not sig or sig in seen_txs:
        continue

    seen_txs.append(sig)

    # Try to get token info from transaction
    tx_detail = get_solana_tx_detail(sig)
    if not tx_detail:
        continue

    # Extract token from transaction
    token_address = None
    amount_sol = 0

    token_transfers = tx_detail.get("tokenTransfers", [])
    sol_transfers = tx_detail.get("solTransfers", [])

    for transfer in token_transfers:
        if transfer.get("toUserAccount") == address:
            token_address = transfer.get("token", {}).get("address")
            break

    if sol_transfers:
        for st in sol_transfers:
            if st.get("fromUserAccount") == address:
                amount_sol = st.get("amount", 0) / 1e9
                break

    if not token_address:
        continue

    # Track for signals
    if token_address not in recent_buys:
        recent_buys[token_address] = []

    recent_buys[token_address].append({
        "wallet": address,
        "name": wallet_name,
        "time": now,

```

```

        "amount_sol": amount_sol
    })

    # Clean old buys (older than 30 min)
    recent_buys[token_address] = [
        b for b in recent_buys[token_address]
        if now - b["time"] < 1800
    ]

    # Send individual alert to all users
    for uid, udata in users.items():
        await send_individual_alert(
            app, int(uid), wallet_name, address, token_address, amount_so
        )

    # Check for signal (2+ wallets)
    buyers = recent_buys[token_address]
    if len(buyers) >= 2:
        buyer_names = list(set([b["name"] for b in buyers]))
        buyer_addresses = list(set([b["wallet"] for b in buyers]))

        # Only send signal once per token per 30 min
        signal_key = f"signal_{token_address}"
        last_signal = recent_buys.get(signal_key, 0)
        if isinstance(last_signal, (int, float)) and now - last_signal >
            recent_buys[signal_key] = now
        for uid in users:
            await send_signal_alert(
                app, int(uid), token_address, buyer_addresses, buyer_
            )

    await asyncio.sleep(0.5)

    save_seen_txs(seen_txs)

async def tracker_job(context: ContextTypes.DEFAULT_TYPE):
    try:
        await check_wallets(context.application)
    except Exception as e:
        logger.error(f"Tracker error: {e}")

# --- COMMANDS ---

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    user_id = str(update.effective_user.id)
    users = load_users()
    if user_id not in users:

```



```
users[user_id] = {"joined": datetime.now().isoformat(), "plan": "free"}
save_users(users)
```

```
vip_count = len(load_vip_wallets())
msg = f"""👋 *Welcome to Sierastracking Bot!*
```

🔍 Automatically tracking `*{vip_count} whale wallets*` for you.

📡 You'll receive:

- 📦 Individual alerts for every buy
- 🚨 Signal alerts when 2+ wallets buy same token

`*Commands:*`

```
/status - Bot status
/signals - Recent signals
/recent - Last transactions
/topwallets - View tracked wallets
/help - All commands"""
```

```
await update.message.reply_text(msg, parse_mode="Markdown")
```

```
async def status(update: Update, context: ContextTypes.DEFAULT_TYPE):
```

```
    vip_wallets = load_vip_wallets()
    users = load_users()
    msg = f"""📊 *Bot Status*
```

✅ Running

👥 Tracked Wallets: `*{len(vip_wallets)}*`

👤 Total Users: `*{len(users)}*`

🕒 Check Interval: Every 2 minutes

🕒 Last Check: Just now"""

```
await update.message.reply_text(msg, parse_mode="Markdown")
```

```
async def topwallets(update: Update, context: ContextTypes.DEFAULT_TYPE):
```

```
    wallets = load_vip_wallets()
    msg = f"🐳 *Tracked Whale Wallets ({len(wallets)} total)*\n\n"
    for i, w in enumerate(wallets[:30], 1):
        name = w.get("name") or f"Wallet {i}"
        addr = w.get("address", "")
        short = f"{addr[:6]}...{addr[-4:]}"
        msg += f"`{i}` *{name}* - `{short}`\n"
    if len(wallets) > 30:
        msg += f"\n...and {len(wallets)-30} more wallets_"
    await update.message.reply_text(msg, parse_mode="Markdown")
```

```
async def signals(update: Update, context: ContextTypes.DEFAULT_TYPE):
```

```
    active = {k: v for k, v in recent_buys.items()}
```

```

        if isinstance(v, list) and len(v) >= 2}
if not active:
    await update.message.reply_text(
        "📡 *No signals detected yet.*\n\nSignals fire when 2+ tracked wallet
        parse_mode="Markdown"
    )
    return
msg = "🔴 *Recent Signals:*\n\n"
for token, buyers in list(active.items())[:5]:
    short = f"{token[:6]}...{token[-4:]}"
    names = list(set([b["name"] for b in buyers]))
    msg += f"🔴 `{short}` - *{len(names)} wallets*\n"
    for n in names[:3]:
        msg += f"    • {n}\n"
    msg += f"    [View] (https://dexscreener.com/solana/{token})\n\n"
await update.message.reply_text(msg, parse_mode="Markdown", disable_web_page_

async def recent(update: Update, context: ContextTypes.DEFAULT_TYPE):
    seen = load_seen_txs()
    if not seen:
        await update.message.reply_text("📄 No transactions recorded yet. Waiting
        return
    await update.message.reply_text(
        f"📄 *Recent Activity*\n\n✅ Monitoring active\n📝 Total transactions seen
        parse_mode="Markdown"
    )

async def help_cmd(update: Update, context: ContextTypes.DEFAULT_TYPE):
    msg = ""
    📖 *Commands:*

/start - Start bot
/status - Bot status
/topwallets - View all tracked wallets
/signals - Active signals
/recent - Recent activity
/upgrade - Get PRO plan

*PRO Plan - 20 USDT:*
✅ Add your own custom wallets
Payment: `TNholuvQNYz4gMTj1r1zDxWKNAuzaeZw4W` (TRC20)
Then: /verify_payment <tx_hash>""
        await update.message.reply_text(msg, parse_mode="Markdown")

async def upgrade(update: Update, context: ContextTypes.DEFAULT_TYPE):
    msg = ""
    💎 *PRO Plan - 20 USDT*

✅ Add unlimited custom wallets

```

- ✓ All 100+ VIP wallets (already free!)
- ✓ Signal alerts
- ✓ Safety analysis

 *Payment:*

Send *20 USDT* (TRC20) to:

`TNholuvQNyz4gMTj1r1zDxWKNAuzaeZw4W`

Then send: `/verify\_payment <your\_tx\_hash>`

```

    await update.message.reply_text(msg, parse_mode="Markdown")

async def verify_payment(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not context.args:
        await update.message.reply_text("Usage: /verify_payment <tx_hash>")
        return
    tx_hash = context.args[0]
    user_id = update.effective_user.id
    await update.message.reply_text(
        f"✓ Payment submitted!\nTX: `{tx_hash}`\n\nAdmin will verify and activate"
        parse_mode="Markdown"
    )
    if ADMIN_ID:
        await context.bot.send_message(
            ADMIN_ID,
            f"$ New payment verification!\nUser: {user_id}\nTX: {tx_hash}"
        )

async def grant_pro(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if update.effective_user.id != ADMIN_ID:
        return
    if not context.args:
        await update.message.reply_text("Usage: /grant_pro <user_id>")
        return
    target_id = context.args[0]
    users = load_users()
    if target_id not in users:
        users[target_id] = {}
    users[target_id]["plan"] = "pro"
    save_users(users)
    await update.message.reply_text(f"✓ PRO activated for user {target_id}")
    try:
        await context.bot.send_message(int(target_id), "🎉 Your PRO plan is now active")
    except:
        pass

async def watch(update: Update, context: ContextTypes.DEFAULT_TYPE):
    user_id = str(update.effective_user.id)

```

```

users = load_users()
user = users.get(user_id, {})

if user.get("plan") != "pro":
    await update.message.reply_text(
        "🔒 Custom wallet tracking requires PRO plan.\n\nType /upgrade to lea
        parse_mode="Markdown"
    )
    return

if not context.args:
    await update.message.reply_text("Usage: /watch <wallet_address>")
    return

address = context.args[0]
await update.message.reply_text(f"✅ Now tracking: `{address}`", parse_mode="

def main():
    if not TOKEN:
        print("ERROR: TELEGRAM_BOT_TOKEN not set!")
        return

    app = Application.builder().token(TOKEN).build()

    app.add_handler(CommandHandler("start", start))
    app.add_handler(CommandHandler("status", status))
    app.add_handler(CommandHandler("topwallets", topwallets))
    app.add_handler(CommandHandler("signals", signals))
    app.add_handler(CommandHandler("recent", recent))
    app.add_handler(CommandHandler("help", help_cmd))
    app.add_handler(CommandHandler("upgrade", upgrade))
    app.add_handler(CommandHandler("verify_payment", verify_payment))
    app.add_handler(CommandHandler("grant_pro", grant_pro))
    app.add_handler(CommandHandler("watch", watch))

    job_queue = app.job_queue
    job_queue.run_repeating(tracker_job, interval=120, first=30)

    print("🚀 Bot started!")
    app.run_polling(drop_pending_updates=True)

if __name__ == "__main__":
    main()

```